Real-Time Schedule Compliance via Spatiotemporal Predicate Evaluation and Relational Lookup

Dr. S. Suresh Kumar / ScholarMaster Research Group
Swarnandhra College of Engineering & Technology (Autonomous)

Abstract*—Sensing infrastructure detecting spatial presence is becoming autonomous from institutional schedule databases, making it challenging to ensure observations of presence correspond to required obligations. This work introduces a relational evaluation framework which constantly harmonizes detected spatial presence with institutional schedules using predicate logic. The framework introduces the Continuous Predicate Evaluation Model (CPEM) which encodes institutional rules as logical predicates over continuous streams of event-tuples. A Persistent Constraint Violation Filter (PCVF) is used to suppress transient sensor-induced and transit-related constraint violations that arise due to temporal instability of state transitions. This is achieved through persistent violation accumulation before allowing any state transition to occur. Additionally, spatial exclusion constraints prevent simultaneous detections within proximity zones which could lead to contradictory rule evaluations. To ensure that bursty traffic during context switches between classes is handled correctly, the relational pipeline caches connections via transaction-level pooling, partitions tables using list-based partitioning and composite time-based indexes in PostgreSQL. Evaluation layer load-testing confirms that at around 5000 queries per second (QPS) with less than 15 ms p99 latency from evaluation layer is achievable under bursty loads, thus continuous schedule reconciliation can be maintained within acceptable evaluation latency by optimizing relational access patterns.

Keywords*—*Continuous Predicate Evaluation, Spatiotemporal Compliance, Relational Stream Processing, Temporal Debounce Logic, Burst-Resilient Query Optimization.

\begin{abstract}

Sensing infrastructure detecting spatial presence is becoming autonomous from institutional schedule databases, making it challenging to ensure observations of presence correspond to required obligations. This work introduces a relational evaluation framework which constantly harmonizes detected spatial presence with institutional schedules using predicate logic.

The framework introduces the Continuous Predicate Evaluation Model (CPEM) which encodes institutional rules as logical predicates over continuous streams of event-tuples. A Persistent Constraint Violation Filter (PCVF) is used to suppress transient sensor-induced and transit-related constraint violations that arise due to temporal instability of state transitions. This is achieved through persistent violation accumulation before allowing any state transition to occur. Additionally, spatial exclusion constraints prevent simultaneous detections within proximity zones which could lead to contradictory rule evaluations.

To ensure that bursty traffic during context switches between classes is handled correctly, the relational pipeline caches connections via transaction-level pooling, partitions tables using list-based partitioning and composite time-based indexes in PostgreSQL. Evaluation layer load-testing confirms that at around 5000 queries per second (QPS) with less than 15 ms p99 latency from evaluation layer is achievable under bursty loads, thus continuous schedule reconciliation can be maintained within acceptable evaluation latency by optimizing relational access patterns.

\end{abstract}

\begin{IEEEkeywords}

Continuous Predicate Evaluation, Spatiotemporal Compliance, Relational Stream Processing, Temporal Debounce Logic, Burst-Resilient Query Optimization.

\end{IEEEkeywords}

INTRODUCTION

The main thesis of this paper is that continuously evaluated relational predicates are crucial to achieving bounded latency reconciliation between spatial event streams and institutional schedules in the face of bursting concurrent accesses. This paper is a relational stream-evaluation study which addresses spatiotemporal logic and query optimization rather than general institutional surveillance questions.

The Temporal Blindness of Snapshot Attendance Systems

Traditional attendance policies are temporal blind. Single-point location verification (RFID tag or biometrics) assumes future compliance based on a punctual confirmation. But the world changes after verification – people move, miss the bus, take early lunch, etc. Temporal verification is not the same as continuous relational validation over events in a timeframe.

Subsystem Scope and Canonical Separation

Within the ScholarMaster architecture, this subsystem reconciles spatial events with schedules, constantly evaluates compliance predicates, stabilizes transient violations, and delivers bounded-latency query response. This subsystem does NOT perform local acoustic anomaly detection (Paper 6), activate and orchestrate policies (Paper 9), manage cryptographic provenance (Paper 8), engage in sensing inference, or perform behavioral analytics.

Continuous Predicate Evaluation

The approach we are proposing for closing the semantic gap between sensing layers and schedule databases fundamentally relies on rethinking the problem of compliance evaluation. Indeed, our evaluation function is based on the notion of presence as a continuous flow, rather than a punctual event. The logic of the stream of tuples is similar to that of continuous relational state, as it has been defined for real-time query processing in [Ref].

Contributions

Implementing continuous compliance evaluation required several adaptations within the relational pipeline. The primary focus is the evaluation logic necessary to make continuous reconciliation viable under institutional-scale burst traffic.

Specifically, this paper:

- Formalizes spatiotemporal compliance rules as continuously evaluated relational predicates (CPEM), enabling schedule comparison at event granularity.
- Introduces a temporally stabilized violation mechanism (PCVF) to suppress short-lived mismatches caused by transit or sensor instability.
- Establishes spatial exclusivity invariants to prevent logically inconsistent upstream detections from contaminating the evaluation layer.
- Optimizes relational read paths using connection pooling, table partitioning, and composite indexing to sustain burst concurrency.

Sensing, identity resolution, and governance escalation are treated as external subsystem dependencies; this work isolates the relational predicate evaluation layer.

RELATED WORK

Limitations of Passive Evaluation

Existing institutional monitoring infrastructure focuses on achieving accurate detection with often limited opportunities for real-time schedule reconciliation, as most are designed to simply react to data rather than analyze it as it comes in. With little to no automatic filtering, many important anomalous events get lost in the stream of information that the human operators have to deal with [Ref], and it becomes unfeasible to manually cross-reference observed events with the ever-changing schedule data.

The approach suggested here is based on an exception-based evaluation. Human operator attention is drawn only when a formally specified deviation is detected beyond the debounce period [Ref], thus lightening the supervisory function without adversely affecting the responsiveness of the process.

Discrete Verification versus Continuous Reconciliation

RFID badges or biometric identifiers can establish presence at a particular checkpoint [Ref]. It does not monitor presence within a given location. Additionally, once a student is marked present, their status is usually not changed unless it is manually adjusted.

A continuous reconciliation over a zone as described in this paper goes much further than the preceding model. By using ideas from moving-object databases [Ref], it performs reconciliation at every instant during the interval. Whereas the preceding approach reconciles a single predicate at one point in time, the continuous zone-based approach reconciles an ongoing stream of predicates.

Stream Processing and Spatiotemporal Coordination

Recent work on edge-intelligent systems has focused on dense IoT integration for operational visibility [Ref]. Multimodal fusion approaches have seen progress in situational awareness over the target domain [Ref]. The current work similarly seeks to leverage cross-relational coordination to improve schedule adherence, but with an emphasis on detection events [Ref] in conjunction with centralized relational records.

The emphasis here is on relational reconciliation rather than perceptual optimization—a distinction that positions this work closer to stream-processing systems research [Ref] than to computer vision or behavioral analytics.

OPERATIONAL MODEL

Context-Blindness in Edge Sensors

Edge detection systems are usually designed for perception-level policies or decisions. They can recognize the opaque identifier token and relate it to a specific region at a particular moment. However, they cannot decide if the presence is authorized compared to a few specific rules [Ref].

Consider two identical physical observations at 14:30:

- Scenario A: Student X is detected at the Exit Gate. The schedule indicates campus dismissal. \rightarrow Evaluation: Compliant.
- Scenario B: Student X is detected at the Exit Gate. The schedule indicates mandatory laboratory. \rightarrow Evaluation: Non-compliant.

From the sensor's point of view, these are not different situations. The difference is introduced when the spatial event has to be related to the schedule. The evaluation of predicates yields the formal state of compliance.

Continuous Predicate Evaluation Model (CPEM)

Unlike the one-off decision to simply attend a lecture as in the initial compliance model, we think of compliance as a state which exists or not over time; the CPEM captures this by transforming schedule information into continuously true propositions which are checked against the stream of events. Let us define the stream of sensor events as:

Equation: $E = \{(s_i, z, t)\}$

where:

- s_i denotes an opaque identity token,
- z denotes the detected physical zone,
- t denotes the timestamp.

Compliance is evaluated by comparing observed location $\text{Loc}(s_i, t)$ with the scheduled expectation $S(s_i, t)$. We define the deviation predicate:

Equation: $T(s_i, t) \iff \text{Loc}(s_i, t) \neq S(s_i, t) \land S(s_i, t) \notin \{\emptyset, \text{Break}\}$

If the schedule is empty (e.g. free period or transit window), then the presence is compliant unless it is within a restricted zone. This transforms a predicate over a point to a relation over a continuously varying set of points.

Spatial Exclusivity Invariant

Robust relational operations require defensive handling of inconsistencies in the upstream information; spatial exclusivity emerges as a general logical consistency check, not only an anomaly. Inconsistencies in spatial locations poison downstream analysis.

We enforce the spatial exclusivity invariant:

Equation: $|\{z : \text{Loc}(s_i,t)=z\}| \le 1$

A token representing an entity cannot reside in two disjoint zones simultaneously. If detections are made for the same object (token) at the same time, the evaluation layer recognizes this as a teleportation anomaly (relationally, it violates the consistency of the state vector) and thus suspends processing for this token until the situation is resolved, avoiding further spurious relations in the relational layer.

ABSTRACT PROCESSING MODEL

To operationalize real-time compliance evaluation, we assume a logical processing model positioned between an incoming event stream and a relational schedule store.

End-to-End Logic

We assume an event stream and a relational store. Event tuples with identity and resolved zones are ingested as inputs. The former are buffered to maintain history for debounce filtering, and the latter are evaluated against a relational database, such as PostgreSQL, the source of truth, to collect expected zones for validation purposes.

Zone Mapping and Resolution

Zone identifiers are pre-assigned in a centralized configuration repository. The resolution logic is given resolved zone labels directly, removing any need for additional geometric calculations. By removing geometric reasoning from the compliance logic, the overall complexity in the relational fusion layer is decreased.

DATA SANITIZATION & DEBOUNCE LOGIC

Persistent Constraint Violation Filter (PCVF)

Sensor-based zone transitions often have transients which can lead to false compliance exceptions. Transients events are introduced due to motion and sensor instabilities, and treating every event as an independent test could lead to an increased number of false positives. PCVF focuses on long-term stability of the logical state, and not just eliminating transients, as operational drift may be more important to detect than momentary mismatches.

Instead, deviation is treated as a time-accumulated state. We define the sustained violation integral:

Equation: $V(s_i,t) = \int_{t_0}^{t} \mathbb{1}_{T(s_i,\tau)} d\tau$

To approximate a rectangular low-pass filter, we compute:

Equation: $\hat{T}(t) = \frac{1}{\tau_{debounce}} \int_{t-\tau_{debounce}}^{t} T(\tau') d\tau'$

In implementation, the integral is discretized over event timestamps. An alert is emitted only when:

Equation: $V(s_i, t) \ge \tau_{debounce}$

Short-lived differences are swallowed up as glitches. Sustained ones cause escalation. The logic keeps an in-memory LRU cache of the most recent violations to avoid trampling the database too often.

Hysteresis Handling

If an identity is temporarily removed from the feed (e. g., occlusion) and then reappears within the hysteresis window in the same zone, the violation integral is accumulated continuously while the identity is not tracked. If the identity is reassociated with a different zone, the violation integral is reset.

This state-machine behavior (Figure \ref{fig:fsm}) stabilizes evaluation across intermittent signal loss while preserving responsiveness to genuine zone changes.

[FIGURE: Architectural Pipeline / Validation Curves]

}

\caption{PCVF State Machine acting as temporal low-pass filter. Student triggered out-of-schedule transition to Transient state. Alert will only be issued if accumulated violation integral reaches debounce threshold.}

\label{fig:fsm}

\end{figure}

\begin{algorithm}[htbp] % <— CHANGED FROM [H] TO FIX SPACING

\caption{Temporal Debounce Filtering (PCVF)}

\begin{algorithmic}[1]

\Require Incoming Event Stream S, Threshold \tau_{debounce}

\Ensure Validated Logic Events E

\For{each event e in S}

\State ID \leftarrow e.identity

\State Zone \leftarrow e.zone

\State StateBuffer[ID].append(Zone, now())

\State

\State // Check contiguous duration in current zone

\If{\text{Duration}(StateBuffer[ID] == Zone) \ge \tau_{debounce}}

\State E.add(ID, Zone) \Comment{Promote to stable event}

\Else

\State Drop \Comment{Filter as transient noise}

\EndIf

\EndFor

\State \Return E

\end{algorithmic}

\label{alg:sanitization}

\end{algorithm}

BURST-ELASTIC READ OPTIMIZATION

Synchronized class transitions impose brief, extremely high event traffic. Latency is reduced by controlled relational-access behavior rather than intrinsically new database theory. Throughput is gained by decreasing relational access-paths, since pooling reduces concurrency pressure and partitioning decreases lookup scope.

Schema Partitioning Strategy

The student_schedule table is partitioned by day_of_week, immediately reducing search scope to a single partition. This reduces unnecessary cross-day scanning and constrains index traversal depth.

[FIGURE: Architectural Pipeline / Validation Curves]

When a stable event passes the PCVF layer, the logic issues a point-in-time query:

\begin{lstlisting}[language=SQL]

SELECT expected_zone_id FROM student_schedule

WHERE student_id = ? AND day_of_week = ?

AND start_time <= ? AND end_time >= ?

\end{lstlisting}

Composite Temporal Indexing

A composite B-Tree index [Ref] is applied across identity and temporal fields:

\begin{lstlisting}[language=SQL]

CREATE INDEX idx_temporal_lookup ON

student_schedule (student_id, day_of_week, start_time, end_time);

\end{lstlisting}

For a given student's schedule cardinality (limited), this index prunes the candidate intervals quickly before performing any temporal comparisons [Ref]. Most of the time is spent in the network and in I/O, rather than in the index.

Connection Pooling and Capacity Stabilization

During burst transitions, connection setup overhead can dominate query time. Without pooling:

Equation: T_{query} = T_{exec} + T_{conn}

When T_{conn} becomes significant, effective service capacity (\mu) collapses under high arrival rate (\lambda).

Transaction-level connection pooling removes this overhead from the critical path:

Equation: T_{query} \approx T_{exec}

This data architecture adjustment significantly increases sustainable throughput and prevents connection exhaustion during thundering herd scenarios [Ref].

EXPERIMENTAL EVALUATION

Evaluation Scope and Environment

Evaluation was done using two primary criteria: lookup responsiveness and stability under bursty loads. All results were obtained from synthetic load testing, meant to stress-test logical correctness and throughput of the database layer. The intent was not to benchmark realistic user demographics, but to evaluate the relational layer under controlled loads.

Tests were executed on:

- 16-core AMD EPYC server
- 64 GB RAM
- PostgreSQL 15
- shared_buffers = 8GB

Each experiment was repeated five times with randomized identity distributions to smooth out skew from repeated key reuse patterns.

Latency Decomposition

Real-time responsiveness represents an important operational requirement. Rather than reporting a single aggregate metric, we decomposed latency into its constituent components.

TABLE: Latency Decomposition per Event (Single Query)

Most of the latency is incurred in the relational lookup stage. The predicate evaluation contributes a relatively small amount to the overall latency, compared to the cost of networks and IO. This demonstrates that the complexity of the compliance layer has relatively little impact on the response time of the system, once the relational access path has been optimized. Even at higher levels of concurrency, the evaluation layer showed sub-millisecond latency.

Throughput and Concurrency Stress Testing

To evaluate burst resilience, the database was flooded with pre-generated identity tokens representing synthetic institutional burst conditions. Load was increased incrementally to 6,000 QPS.

Without connection pooling, the system began to degrade past 3,000 QPS. Tail latency rose sharply as connection overhead accumulated, confirming the expected T_{conn} bottleneck.

With transaction-level pooling enabled, the latency curve becomes noticeably flatter: within the measuredburst conditions and given the database setup, the service processes about 5,000 queries per second with a sub-15 ms p99 latency (\pm 1.2 ms standard deviation). How many QPS will be supported depends on the actual hardware and network infrastructure one deploys the service on: these are just measurements made against particular workloads that are used as a reference.

[FIGURE: Architectural Pipeline / Validation Curves]

\caption{Throughput Stress Test - without pooling (Red): connection exhaustion leads to p99 latency spiking beyond 3000 QPS. With connection pooling (Green): 5000 QPS with p99 latency below 15 ms.}

\label{fig:throughput}

\end{figure}

The main point was not even the absolute maximum throughput but the stability under synchronized events. In real-life scenarios, when classes end at the same time on a campus, there will be a burst of traffic that requires special handling. Thus, tail latency is more important than absolutely high throughput, and the relational setup sustained it without spikes that would cause cascading failures or connection resets.

Scenario-Based Operational Testing

In addition to synthetic throughput tests, we evaluated specific behavioral scenarios to ensure logical correctness under edge conditions.

[FIGURE: Architectural Pipeline / Validation Curves]

[FIGURE: Architectural Pipeline / Validation Curves]

Scenario 1: Rapid Multi-Zone Transitions\\

An agent traversed multiple restricted zones within a short time interval.

Result: The PCVF state machine absorbed these transitions as transient noise. Because the violation integral never exceeded \tau_{debounce}, no alert was emitted. This confirmed that transit behavior does not trigger false escalation.

Scenario 2: Intermittent Signal Loss\\

An agent's signal was deliberately interrupted for two seconds before being returned to the same zone. As a result, the hysteresis buffer maintained the violation status from before the interruption. Thus, when the signal was restored, the debounce timer did not reset but instead continued counting down. Consequently, the continuous violation was still detected.

Scenario 3: Backend Service Interruption\\

The database connection was forcibly closed while the command was executing. The logic gracefully handled the exception by capturing it allowing the upstream event queue to accumulate undelivered events. After reconnecting, the queue was emptied, delivering all events without data loss.

These controlled tests suggest that the evaluation logic remains stable under common operational disruptions.

DISCUSSION

Logic Decoupling and Extensibility

The value of separating the evaluation in relational predicate logic from the means of establishing identity is that the evaluation remains independent of how identification takes place. Using optical recognition, RFID badges, WiFi triangulation, or some other yet to be invented means, the same tuple could be produced:

Equation: (ID, Zone, Timestamp)

As long as that abstraction is honored, the relational predicate layer is essentially generic in its function, independent of the particular upstream sensing technology. In addition, this decoupling offers significant value in terms of logic evolution: should the quality of the sensing technology be improved, the evaluation query need not be altered; and new compliance rules can be expressed as additional predicates.

Predicate Auditability

A pragmatic benefit of continuous relational evaluation is that every compliance decision is auditable by construction. For every event that triggered a violation, the system can report the tuple that caused it, the schedule that it was checked against, the PCVF integration at the time of escalation, and the spatial exclusivity decision. This traceability is a structural property of predicate-based evaluation rather than a post-hoc logging feature.

In contexts where the compliance decision making process of an institution can be subject to administrative reivew, there is value in being able to reconstruct the decision making path taken by the model. This capability is generally not available when using black box behavioral inference models.

Limitations

The current system only takes into account the relations of the schedule in terms of being in or out of a zone. The system does not make any behavioral predictions beyond that. In doing so, the system maintains logical integrity and evaluation time constraints while limiting the scope of the analysis.

All the results were obtained using synthetic traffic loads, as described in the previous section, and do not reflect the true production traffic from institutions that would be using such a system. It should be noted that while the test load profiles are close approximations to bursts seen during class transitions, deploying the system in a live site will see additional variations on the traffic patterns. These include network jitter, transactions during overlapping transitions, propagation delays during schedule updates, clock drifts of sensors, and temporary backend backend downtimes.

Moreover, the evaluation layer is completely dependent on the upstream sensing quality for the PCVF: if the sensing sub-system incorrectly reports delayed or wrong labels for any of the zones, the PCVF will erroneously interpret some of the true transitions as spurious events. The debounce period \tau_{\text{debounce}} value is therefore a delicate trade-off between suppressing noise-induced transients and retaining responsiveness to legitimate events.

CONCLUSION

This relational stream-evaluation study proposed a novel framework for continuous reconciliation of spatial detection events with institutional schedule constraints formalized through predicate logic. The solution represented institutional rules as continuously evaluated spatiotemporal predicates (CPEM) and applied a temporally stabilized violation filter (PCVF) in order to distinguish

persistent deviations from transient sensor noise while maintaining relational state coherence.

From an infrastructural viewpoint, the reduction in latency is achieved through the limitation of relational-access behavior. The combination of a partitioned schema design, composite temporal indexing and connection pooling at the transaction level enable the relational layer lookup to handle burst-like traffic of the scale of 5000 QPS while still delivering on sub-15 ms p99 latency, thus proving the overhead incurred by relational coordination is acceptable under the tested burst conditions.

The design emphasizes its role in managing predictable processes rather than providing broad analytical functionalities. For the overall functionality of ScholarMaster, this sub-system is only responsible for checking continued adherence to certain criteria and performing relevant, limited-latency relational operations. In other words, it does not control activation governance (Paper 9) and does not engage in auditory anomaly recognition (Paper 6). It takes abstracted spatial event tuples as input and produces states of relevant predicates as output, focusing on offering temporal stability and burst resistance rather than general surveillance.

REFERENCES

[1] D. Comer, "The ubiquitous B-tree," ACM Computing Surveys (CSUR), vol. 11, no. 2, pp. 121-137, 1979.

[2] D. L. Hall and J. Llinas, "An introduction to multisensor data fusion," Proceedings of the IEEE, vol. 85, no. 1, pp. 6-23, 1997.

[3] B. Salzberg and V. J. Tsotras, "Comparison of access methods for time-evolving data," ACM Computing Surveys (CSUR), vol. 31, no. 2, pp. 158-221, 1999.

[4] R. H. Güting et al., "A foundation for representing and querying moving objects," ACM Transactions on Database Systems (TODS), vol. 25, no. 1, pp. 1-42, 2000.

[5] M. Stonebraker et al., "The 8 requirements of real-time stream processing," SIGMOD Record, vol. 34, no. 4, pp. 42-47, 2005.

[6] E. Wu et al., "High-performance complex event processing over streams," SIGMOD, pp. 407-418, 2006.

[7] J. M. Hellerstein et al., "Architecture of a Database System," Foundations and Trends in Databases, vol. 1, no. 2, pp. 141-259, 2007.

[8] S. Harizopoulos et al., "OLTP through the looking glass, and what we found there," SIGMOD, pp. 981-992, 2008.

[9] P. K. Atrey et al., "Multimodal fusion for multimedia analysis: A survey," Multimedia Systems, vol. 16, no. 6, pp. 345-379, 2010.

[10] X. Wang, "Intelligent multi-camera video surveillance: A review," Pattern Recognition Letters, vol. 34, no. 1, pp. 3-19, 2013.

[11] D. Abadi et al., "The Beckman Report on Database Research," SIGMOD Record, vol. 43, no. 3, pp. 61-70, 2014.

[12] T. Akidau et al., "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing," Proc. VLDB Endow., vol. 8, no. 12, pp. 1792-1803, 2015.

[13] Y. Zheng, "Trajectory data mining: an overview," ACM Transactions on Intelligent Systems and Technology (TIST), vol. 6, no. 3, pp. 1-41, 2015.

[14] P. Carbone et al., "Apache Flink: Stream and Batch Processing in a Single Engine," IEEE Data Engineering Bulletin, vol. 38, no. 4, pp. 28-38, 2015.

[15] W. Shi et al., "Edge Computing: Vision and Challenges," IEEE Internet of Things Journal, vol. 3, no. 5, pp. 637-646, 2016.

[16] M. Satyanarayarayanan, "The Emergence of Edge Computing," Computer, vol. 50, no. 1, pp. 30-39, 2017.

[17] S. Bhattacharya, G. S. Nainala, P. Das, and A. Routray, "Smart attendance monitoring monitoring system (SAMS)," in ICALT, IEEE, 2018, pp. 358-360.

[18] Z. Zhou et al., "Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing," Proceedings of the IEEE, vol. 107, no. 8, pp. 1738-1762, 2019.

[19] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. McGraw-Hill, 2019.

[20] N. Min-Allah and S. Alrashed, "Smart campus: A sketch," Sustainable Cities and Society, vol. 59, p. 102231, 2020.

[21] S. Dev and T. Patnaik, "Student Attendance System using Face Recognition," in ICOSEC, IEEE, 2020, pp. 1-6.

[22] L. Zhang, "Spatio-Temporal Visual Analysis for Urban Traffic Characters Based on Video Surveillance Camera Data," ISPRS International Journal of Geo-Information, vol. 10, no. 3, p. 177, 2021.

[23] M. A. Khan et al., "Video Surveillance Anomaly Detection: A Review on Deep Learning Benchmarks," IEEE Access, vol. 12, pp. 1-15, 2024.

[24] S. Suresh Kumar, "Perception Integrity Foundations: Evidential Uncertainty and Calibrated Disagreement in Edge Vision," ScholarMaster Research Series, Paper 22, 2026.

[25] S. Suresh Kumar, "ScholarMaster Integration Architecture and Downstream Error Propagation Analysis," ScholarMaster Research Series, Paper 25, 2026.

[26] S.~Suresh~Kumar, "ScholarMaster macro integration architecture and downstream error propagation analysis," \emph{ScholarMaster Technical Report Series}, Paper 25, 2026.